Programmable Tuple Spaces for Mobile Agent Coordination

Franco Zambonelli
University of Modena and Reggio Emilia – ITALY

franco.zambonelli@unimo.it
http://www.dsi.unimo.it/Zambonelli

Outline

- Interactions in mobile agent applications
  - Taxonomy
  - Comparison of Models
- Programmable tuple spaces
- The MARS coordination architecture
- Application example

Internet Applications

- The Internet no longer used as a simple
  - communication infrastructure OR
  - information repository
- The Internet as a *global distributed computing system*
  - traditional distributed applications executed at a world-wide scale
  - new application areas
- Communication, computation and data management intertwined in several application areas
  - E-commerce
  - Information Retrieval
  - CSCW

Internet Agents

- Autonomous (*task-oriented*)
  - execute in *autonomy* to achieve a *goal*
- Network-aware (*location-aware*)
  - conscious about the *distribution* of the environment
- Mobile
  - can proactively roam through Internet sites (changing their *execution environment*)
  - mobile devices can be abstracted as mobile Internet agents
- Interactive
  - inter-agent interactions
  - agent-environment interactions

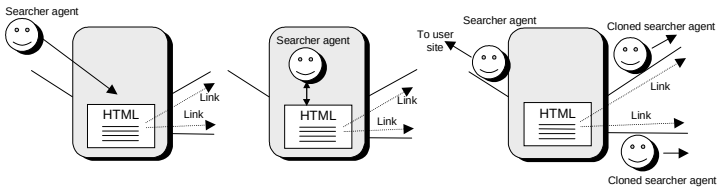
Interaction Models: a Taxonomy

- Temporal uncoupling
  - no activity synchronization
  - no time co-presence
- Spatial/Name uncoupling
  - no mutual knowledge (*e.g.* name identifier)

Spatial \ Temporal	Coupled	Uncoupled
Coupled	Direct *Aglets* *D'Agents*	Blackboard-Based *Ambit* *ffMain*
Uncoupled	Meeting-Oriented *Ara* *MOLE*	Linda-like *PageSpace* *TuCSoN* *MARS*

Case Study: Distributed Information Retrieval

- Agents reach remote sites to access and analyze HTML pages
- Cloning to follow remote links (possibly interesting) ⇨ dynamic tree of mobile agents
- After a visit, agents come back with the URLs of interests (*e.g.* matching a keyword)



Interaction in the Case Study

- With the local execution environment:
  - retrieve info on local HTML files
- Facts:
  - require access control
  - heterogeneity of execution environments
- Inter-agent
  - avoid duplicated visits on the same sites (due to cross-references in HTML pages)
- Facts
  - agents are dynamically created
  - cannot foresee their number and location

Case Study: Sample Code in Aglets *(simplified)*

```
Public class ExplorerAgent extends Aglet {
boolean _clone = false;     boolean visited = false;                String keyword = null;
URL destination = null;     URL home_URL = "atp://my.site"     Vector new_destinations;
….
public void onCreation(Object args) {
   destination = (URL)((Object[]) args)[0]; keyword = (String)((Object[]) args)[1];
   dispatch(destination);}
….
public void run()
{   // CHECKS IF ALREADY VISITED:        INTER-AGENT COORDINATION
   if(visited) dispatch(home_URL);  // the agent has nothing to do on this site
   // RETRIEVES LOCAL HTML FILES:      AGENT-ENVIRONMENT COORDINATION
   // search for the keyword in the HTML files and for other interesting sites
   // for instance, is a Vector of references to HTML files has been retrieved:
            If (HTMLFiles.search_keyword(keyword)) …..
            new_destinations = this.extractUniqueURLs(HTMLFiles); …..
   // create clones and send them to the computed destinations
   for(int i=0; i < destinations.size(); i++)   {
            clone();
            if(_clone) // the  _clone variable can be set to true in the OnClone() method
                     dispacth((URL)new_destinations.elementAt(i);} // goes to a new destination
   dispatch(home_URL); // the original agent comes back home
}
```

Direct Interaction Models

- Peer-to-peer agent communication (message-passing)
- Client-server access to services (RPC, RMI)
- Systems: most of Java-agent systems (*Aglets*, *D'Agents*, …)
- Spatial coupling
  - entities have to know each other to interact
- Temporal coupling
  - synchronization of the activities during interaction

Direct Interaction: Pro & Cons

- Advantages
  - well-known and understood model
  - several tools available
  - sometimes very efficient
- Drawbacks
  - how to deal with agent unpredictability
    - e.g.: mobility (tracing, forwarding, only-once messages, performances)
  - how to deal with security
    - agent exchange message without mediation
  - how to scale up communication protocols & patterns
    - combinatorial explosion of communication channels

Case Study: Direct Interaction *(1)*

- Access to local resources via a WWW server
  - should the agent explicitly navigates to all local links to retrieve all HTML files?
  - complex solution!

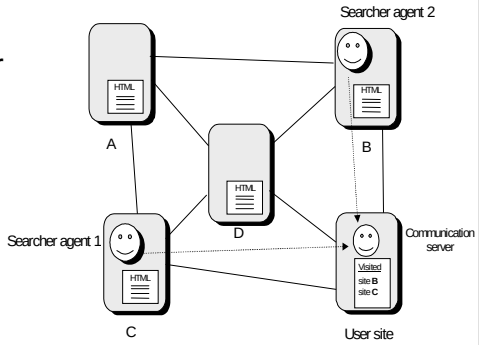
```
URL starting_point = new URL ("www.starting.site");
Vector other_local_HTML_files = recursively_parse(starting_point);
// the code of the recursively_parse method may be rather complex and time
consuming….
```


Case Study : Direct Interaction *(2)*

- Avoiding multiple visits
- Application-level definition of a specialized communication server
  - agents ask to it about previous visits to a site
- Drawbacks
  - centralisation
  - unreliability

```
Message msg = new Message(my_id, today, local_URL);
AgletProxy com_ser = getAgletContext.getAgletProxy(URL,
                                                   com_ser_id);

visited = com_ser.sendMessage(msg);
```



Meetings

- "*Agora*" for agent interactions
- meetings are opened by application agents (always-open meetings abstract the role of services)
- an agent can join a meeting and there communicate with other entities in the meeting
- Systems: *Telescript*, *Ara*, *MOLE* (to some extent)
- Spatial uncoupling
  - only meeting point must be known to enable interactions
- Temporal coupling
  - agents must be on the same place at the same time to interact

Meetings: Pro & Cons

- Advantages
  - locally constrained meetings lead to predictable performances in interactions
  - meetings mediate all interactions and achieve good application-level control over them
- Drawbacks
  - limitation of the agents' autonomy
  - who, where and when to open meeting points for an application?

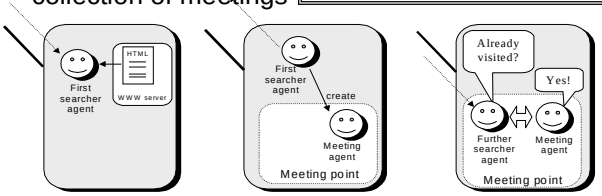
Case Study: Meetings *(1)*

- Access to local data
  - An always-open meeting on a site can abstract a local WWW service
- Avoiding multiple visits
  - Other agents must explicitly open meeting point to get notified of already visited sites
  - Where to create meeting point?
    - one site: centralisation
    - selected given sites: requires agent knowledge
    - all visited sites: spamming of meetings

Case Study: Meetings *(2)*

- The first agent on a node creates a meeting point
- Further agents arriving on that site will enter the meeting point and will get notified of a previous visit
- Advantages
  - fully decentralised
- Drawbacks
  - security and garbage collection of meetings

```
//aglet-like code for meetings
Message msg = new Message(my_id, today);
meet_ag =context.Meet(meeting_agent_id);
if(meet_ag == null)
{  meet = context.createMeeting(meeting_classURL,
                                meeting_agent_id);
   meet.StoreMessage(msg);     }
else   visited = meet_ag.sendMessage(my_id, today);
```



Blackboard-based Interaction

- Each node/domain of nodes should provide for a shared information space to be used by agents to store and retrieve information
- Wherever agents are, they can exploit the blackboard to communicate with other application agents
- Systems: *ffMAIN* (http-based), *Ambit* (file-based)
- Temporal uncoupling
  - messages can be left on and retrieved from blackboards at any time
- Spatial coupling:
  - requires agreement on message identifiers (e.g., URLs in ffMAIN, filenames in Ambit)

Blackboards: Pros & Cons

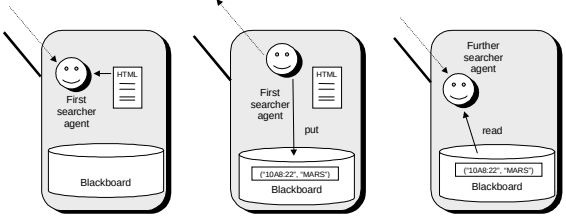
- Advantages
  - temporal uncoupling make it easier for agent to communicate while preserving their autonomy
  - security (all interactions can be monitored by the blackboard)
- Drawbacks
  - spatial coupling increases application complexity due to a priori agreements on message identifier
  - complex interaction protocols may be difficult to implement
  - requires a specific infrastructure for the Internet

Case Study: Blackboards *(1)*

- Access to local data
  - A blackboard can publish data about its public files and information on locally available services
  - Requires a standard way to identify information
  - How can agents selectively retrieve HTML files only?
- Drawbacks
  - complex design w.r.t. the access to local data and services

Case Study: Blackboards *(2)*

- Avoiding multiple visits
  - the blackboard can be used as a mailbox
  - agents get/put messages to avoid multiple visits
- Agents access the local blackboard to discover the presence of HTML pages in the site
- Advantages
  - simple design w.r.t. inter-agent coordination



Tuple Space Interaction

- Each node/domain of nodes should provide for a tuple space to be used by agents to store and retrieve tuples via a Linda-like interface
- Wherever agents are, they can exploit the local tuple space to communicate with other application agents
- Systems: *JavaSpaces, T Spaces, PageSpace, MARS, TuCSoN, Lime*
- Temporal uncoupling
  - as in the blackboard case
- Spatial coupling
  - associative access to tuples does not require strong a priori agreements or knowledge

Tuple Spaces: Pros & Cons

- Advantages
  - full uncoupling suits both mobility of application components and uncertainty of the Internet environments
  - security (inter-agent interactions can be monitored by the blackboard)
  - simplicity of communication primitives
- Drawbacks
  - requires a specific infrastructure for the Internet

Case Study: Tuple Spaces *(1)*

- Access to local data
  - the tuple space can publish data and information on locally available services
  - associative access eases the retrieval of this information
- Access all data via a single operation!

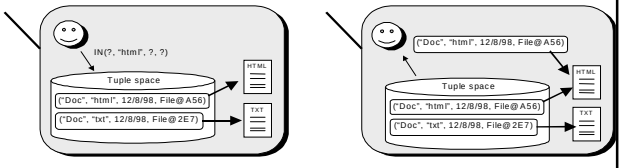
```
HTMLFiles = read_all("Doc","html",date?,name?);
```


Case Study: Tuple Spaces *(2)*

- Avoiding multiple visits
  - agents must check if the tuple space contains tuples left by other agents in a previous visit (similar to the blackboard case)
- Advantages
  - security for interactions
  - very simple agent design

```
if(!(visited = inp(my_application_id, when ?))
      out(my_application_it, System.GetTime());

HTMLFiles = read_all("Doc","html",date?,name?);
```



Summarizing

- Coupled interaction do not suit Internet agents
  - difficult to directly interact at a world-wide scale
  - odd and complex design choices
- Tuple-based models suitable for the Internet
  - uncoupling suits autonomy, mobility and dynamicity
  - associativity suits uncertainty of the Internet
  - can lead to simpler application design
- Actually, it is a *coordination model*
  - clear separation between interaction and computation
  - clear definition of interaction laws
  - shift from *enabling interaction* to *ruling interactions*

What is a Coordination Model?

- A conceptual framework to model the space of interaction
  - Coordinables
    - the entities (agents, processes, objects, components, and so on) whose mutual interaction is ruled by the model
  - Coordination media
    - the abstractions enabling and ruling the interaction among the coordinables
  - Coordination Language
    - the primitives to be used to access the coordination media
  - Coordination laws
    - the rules governing the interaction among coordinables as well as the behaviour of the coordination media

Tuple-based coordination

- Coordinables
  - not fixed by the model
- Coordination media
  - tuple containers “tuple spaces”
- Coordination language
  - read (**rd**), extract (**in**), put (**out**) tuples into tuple spac
- Coordination laws
  - associative access to tuples
    `In(int i?, char ‘c’, float f?)` ***MATCH*** `(5, ‘c’, 3.14)`
  - reactive behaviour
  - synchronization over tuple occurrences

Tuple-based Coordination on the Internet

- Definition of the coordinables
  - several entities involved in applications
  - what the coordinables are?
    - Mobile agents, WWW services, CORBA applications, etc.
- Definition of the coordination media
  - how many? Where?
  - how can they refereed and accessed?
- Definition of Coordination and Communication Languages
  - object-oriented, logic, DBMS oriented models….
- Definition of the coordination laws
  - fixed versus programmable

Programmable Tuple Spaces

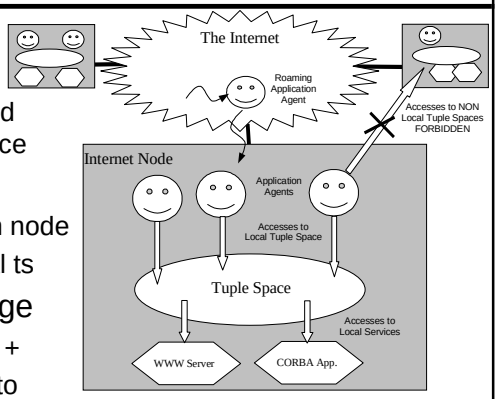
- "Classical" Linda model
  - built-in associative mechanisms
  - lack of flexibility
- Programmable tuple spaces
  - the reactive behaviour of tuple spaces can be programmed
    - not only the pattern-matching mechanism BUT
    - whatever kind of behaviour can be associated to the access events to the tuple spaces
- Why programmable tuple space?
  - flexibility and adaptability
  - clear separation of concerns between coordination laws and algorithm issues
  - monitoring and security

MARS: Mobile Agent Reactive Spaces

- University of Modena, ITALY
- Not yet another agent system BUT
  a coordination architecture
- Ported on different agent systems (*Aglets*, *Java2Go*, *SOMA*)
- Linda-like access to local resources
- Linda-like inter-agent coordination
- Meta-level tuple space for programming tuple space behaviour

MARS Features

- Coordinables
  - Java Agents
  - Everything is accessed via the local tuple space
- Coordination media
  - a tuple space on each node
  - implicit access to local ts
- Coordination Language
  - JavaSpaces interface +
  - `readAll`, `takeAll`, to access all matching tuples
- Coordination Laws
  - fully programmable in Java



MARS: Coordination Laws

- Meta-tuples to associate reaction to access events (meta-matching)
  `(Ag-Id, Tuple, OpType, ReactObj)`
- Access the base-level tuple space triggers pattern-matching in the meta-level to look for reactions to execute
- `ReactObj`: Java object with a single method (the reaction itself)
- Example
  `(ag@mo.it, null, read, O1)`
  triggers the reaction method of object `O1` *when the agent* `ag@mo.it` *performs a* `read` *operation*

MARS: Security Model

- ACLs on tuples
- Agents associated to *roles*
- *Examples:*
  - reader role: can only read tuples
  - write role: can write and extract tuples
  - manager: can install/deinstall reactions
- Agent-installed reactions
  - affects only those agents of the same applications
- Problem
  - garbage collection of reactions

MARS: Sample Code

```
// tuple definition as in JavaSpaces
class FileEntry extends AbstractEntry
{ public String PathName;  public String Extension;
  public Date ModificationTime; public File ActualFile;
  public FileEntry(String name, String extension, Date modificationTime, File file)
  {             PathName = name; Extension = extension;
                ModificationTime = modificationTime; ActualFile = file;      }   }
….
FileEntry FilePattern = new FileEntry(null, “html”, null, null);
// creation of the template tuple, as in JavaSpaces
Vector HTMLFiles = LocalSpace.readAll(FilePattern, null, NO_WAIT);
// access to the local tuple space. The reference is automatically initialised as the agent
arrives on that node
// returns all matching FileEntry tuple, if any
if (!HTMLFiles.isEmpty())              // if some matching tuples are found
        { for (int i = 0; i < HTMLFiles.size(); i++)    // for each matching tuple
              {             FileEntry    Hfile    =    (FileEntry)HTMLFiles.elementAt(i);
                        // Hfile: tuple representing a  single file
                        if          (this.SearchKeyword(keyword,          Hfile.ActualFile))
                        ……
```

MARS: Example of Reaction

- Heterogeneity of data
  - the site store “htm” files, agents looks for “html” files
  - the reaction can transform the “html” extension required by agents into an “htm” extension

```
// can be associated to readAll operations on FileEntry templates with HTML extension
//via the meta-level tuple:   (HTML2HTM_instance, (null, “html”, null, null), readAll, null)

class HTML2HTM implements Reactivity
{
public Entry[ ] reaction(Space s, Entry Fe, Operation Op, Identity Id)
// no match already occurred if the site has only htm files
{ Fe.Extension = “htm”;                          // modifies the extension of the required files
  return s.readAll(Fe, null, NO_WAIT);           // read FileEntry tuples with HTM extension
}}
```


MARS: Example of Reaction

- If agents attempt to extract a tuple
  - monitor the unauthorised access
  - transform the extraction into a read transparently to agents

```
// can be associated to take operations via: (TransformTakeObj, null, take, null)

class TransformTake implements Reactivity
{
public Entry reaction(Space s, Entry Fe, Operation Op, Identity Id)
{ if(matched(Fe)) { // if a match has been produced
            if (Id.equals(manager)) // check for the identify of the agent performing the operation
                        return s.take(Fe, null, NO_WAIT);// the tuple is deleted from the spac
            else  {SecurityRegister.add(“takeAll”, Fe, Id);   // log the access
                        return Fe }  // the tuple is returned but it not extracted
else return null;           }               // no match has been produced   }}
```

Open Issues

- Security issues
  - Internet asks for secure coordination models
  - coordination and security both aim at ruling interactions!
- Economic issue
  - the Internet will become a global market
  - should I pay for interaction events?
- Software engineering tools
  - patterns for Internet-agent composition
  - can coordination ease composition?
- Standardization
  - Agents in Java or in XML?
  - XML dataspaces?

Tuple-based Coordination: Related Work

	Coordinables	Coordination Media	Coordination Language	Coordination Laws
JavaSpaces	Java Agents	OO referencing	OO (Java) Linda primitives	Fixed (object-matching)
Lime	Abstract mobile components	Implicit access (to agent-own tuple space)	Linda	Fixed (pattern-matching)
MARS	Java mobile agents	Implicit access (to local tuple space)	OO (Java) Linda primitives	Programmable (Java reactions)
PageSpace	Agent + Services	OO referencing	OO (Java) Linda primitives	Fixed (object-matching)
T Spaces	Agent, PDA	OO referencing	Linda + user-defined primitives	Programmable (primitives overriding)
TuCSoN	Heterogeneous (possibly mobile) Internet agents	implicit access (to local tuple space) + URL referencing	Logic-oriented Linda	Programmable (FOL reactions)

Conclusions

- Advantages of coordination models and architectures for Internet applications:
  - to facilitate an engineered approach to Internet application development
  - to lead to a simple and secure application design
- Programmable Tuple Spaces:
  - increased security & flexibility
  - clear separation between coordination and computation
- Several open issues and, of course:
  - need for unifying models and reference architectures
  - need for real-world applications